Detailed Implementation Plan: Advanced Procurement Workflows & Notifications

Overview & Goal Description

This implementation plan addresses four major operational bottlenecks in the current system, providing a robust, highly sophisticated architecture to solve them.

The Problems:

1. **No Support for Split Deliveries**: Currently, approving any Goods Receipt Note (GRN) automatically marks the Purchase Order (PO) as `Delivered`, even if the supplier only delivered half the items.
2. **Inaccurate 3-Way Matching**: The matching engine cannot properly validate multiple invoices against a single PO if the deliveries were split, leading to false "Variance" errors.
3. **Global Notifications**: If one user reads a notification, it disappears or marks as "read" for everyone else in the system.
4. **View-Only Alerts**: Notifications only tell users something happened; they cannot take immediate action (like approving a PO) directly from the notification.

The Sophisticated Solution:

1. **Ledger-Based POs**: POs will act like a ledger. Every line item will track exactly how many units were `ordered`, `delivered`, and `billed`. The PO status will dynamically calculate based on these ledger balances (e.g., `Partially Delivered`).
2. **Line-Level Matching Engine**: The 3-way match will calculate cumulatively. It will look at `(Total Delivered - Total Billed)` to determine if a new invoice is valid, down to the specific line item.
3. **Relational User Notifications**: We will introduce a new `UserNotification` database table to track exactly which user read which notification, keeping everyone's inbox isolated.
4. **Action Dispatch Registry**: Notifications will embed "Action Payloads". A central frontend registry will render actionable buttons (e.g., `[Approve PO]`) right in the notification feed, executing commands without requiring the user to navigate away.

User Review Required

> [!CAUTION]
> **Database Schema Changes & Migration**
> We are adding a new table (`UserNotification`) and new fields to JSON objects in the database. Existing POs and Invoices will need to be treated carefully since they won't have the new `deliveredQty` and `billedQty` fields initialized.

> [!IMPORTANT]
> **Action Idempotency (Preventing Double Actions)**
> When an action like "Approve PO" is taken from a notification by User A, we will mark the notification's action state as `COMPLETED`. When User B sees the notification, the button will be disabled and say "Completed by User A", preventing duplicate approvals.

Detailed Proposed Changes

The implementation will be divided into the following component areas.

Phase 1: Database Schema Updates

[MODIFY] `prisma/schema.prisma`

We need to update the database to support user-isolated notifications and stateful actions.

1. **Create the `UserNotification` Model**:

```
model UserNotification {
  id              String   @id @default(cuid())
  userId          String   // The specific user this notification belongs to
  notificationId  String   // Links to AppNotification
  isRead          Boolean  @default(false)
  readAt          DateTime?
  actionState     String   @default("PENDING") // States: PENDING, COMPLETED, FAILED
  actionResult    String?  @default("") // E.g., "Approved by Manager"
  createdAt       DateTime @default(now())
  updatedAt       DateTime @updatedAt

  @@unique([userId, notificationId]) // Ensures one state record per user per notification
}
```

2. **Update `AppNotification` Model**:
   Remove the global `read` flag and add action properties.

```
model AppNotification {
  id          String   @id @default(cuid())
  type        String
  source      String
  title       String
  message     String
  timestamp   String
- read        Boolean  @default(false)
+ actionType String?  @default("") // e.g., 'APPROVE_PO', 'REVIEW_VARIANCE'
+ actionPayload Json? // { poId: '123', requiredRole: 'manager' }
  entityId    String
  entityType String
  createdAt   DateTime @default(now())
}
```

[MODIFY] `src/types/index.ts`

Update our TypeScript definitions to match the new architecture.

- **`POItem`**: Add `deliveredQty?: number` and `billedQty?: number`.
- **`POStatus`**: Ensure it includes `'Partially Delivered'`.
- **`Invoice`**: Add `matchReport?: Record<string, any>` to store detailed line-by-line variance reports.

Phase 2: Ledger-Based Split Deliveries

[MODIFY] `src/app/api/data/mutate/route.ts` (GRN Approval Engine)

When a user approves a GRN, we will update the PO ledger instead of blindly closing the PO.

Detailed Logic for `APPROVE_GRN`:

1. Fetch the associated `PurchaseOrder` from the database.
2. Iterate through the `GRN.lineItems`. For each line, find the matching `PO.lineItems`.
3. Increment the PO line's `deliveredQty` by the GRN line's `acceptedQty`.
4. **Calculate PO Delivery Status**:
    - Compare the sum of all PO line `deliveredQty` against the sum of all PO line `quantity`.
    - If `Total Delivered == 0`: Status remains `Pending Delivery`.
    - If `Total Delivered > 0` AND `Total Delivered < Total Quantity`: Status becomes `Partially Delivered`.
    - If `Total Delivered >= Total Quantity`: Status becomes `Delivered`.
5. Save the updated PO back to the database.

[MODIFY] `src/components/GRNPage.tsx` (New GRN Modal)

- When creating a new GRN against a `Partially Delivered` PO, the system must auto-calculate the remaining quantity.
- **Logic**: For each PO item, set the default receipt quantity to `Math.max(0, item.quantity - (item.deliveredQty || 0))`.

[MODIFY] `src/components/PurchaseOrdersPage.tsx` (PO Details View)

- Update the Line Items table to show a **Fulfillment Ledger**.
- Add columns for `Ordered Qty`, `Delivered Qty`, and `Remaining Qty`.
- Add a visual progress bar (e.g., `[======] 50/100 Delivered`).

Phase 3: Advanced Multi-Invoice 3-Way Matching

[NEW] `src/lib/matchingEngine.ts`

We will extract the 3-Way Match logic into a highly deterministic, cumulative engine.

Detailed Logic for `runThreeWayMatch(invoice, po, grns)`:

1. Create an `AllowedToBill` map for the PO. For each line item, `AllowedToBill = (deliveredQty || 0) - (billedQty || 0)`.
2. Iterate through `Invoice.lineItems`.
3. **Quantity Check**: If `Invoice Line Qty > AllowedToBill`, flag a `Quantity Variance`.
4. **Price Check**: If `Invoice Line Unit Price != PO Line Unit Price`, flag a `Price Variance`.
5. **Generate Match Report**: Create a detailed JSON object listing every variance found (e.g., "Item A: Over-billed by 20 units").
6. **Update Status**:
    - If no variances: Invoice `matchStatus = 'Full Match'`. Update PO's `billedQty` for each line.
    - If variances exist: Invoice `matchStatus = 'Variance'`. Save the `matchReport` to the Invoice record for the user to review.

[MODIFY] `src/components/InvoicesPage.tsx`

- In the Invoice Details modal, if `matchStatus === 'Variance'`, render the `matchReport` in a clean UI table so the user sees *exactly* which line item caused the error and why.

Phase 4: Event-Driven Direct Actions & Notifications

[NEW] `src/app/api/actions/execute/route.ts`

A centralized backend API to handle actions triggered from notifications.

- **Input**: `{ notificationId, actionType, payload }`
- **Logic**:

1. Verify the `UserNotification.actionState` is `PENDING`. If it's `COMPLETED`, return an error ("Action already taken").
2. Execute the business logic based on `actionType` (e.g., if `APPROVE_PO`, run the PO approval mutation).
3. Mark `UserNotification.actionState = 'COMPLETED'` and `actionResult = 'Approved by CurrentUser'`.

[MODIFY] `src/components/Notifications/ActionRegistry.tsx` (New Frontend Component)

A UI component that reads a notification's `actionType` and renders the correct button.

- If `actionType === 'APPROVE_PO'`: Renders a green `[Approve PO]` button. When clicked, it calls the execute API and shows a loading spinner.
- If `actionType === 'REVIEW_VARIANCE'`: Renders a `[Inspect Variance]` button. When clicked, it opens the Invoice Details modal directly.
- If `actionType === 'REORDER_STOCK'`: Renders a `[Create PO]` button. When clicked, it opens the New PO Modal pre-filled with the stock item.

[MODIFY] `src/components/Sidebar.tsx` & `NotificationsPage.tsx`

- Refactor the fetch logic to query `UserNotification` where `userId = currentUser.id`.
- The Unread badge count will now accurately reflect *only* the current user's unread notifications.
- Inject the `ActionRegistry` component inside each notification card.
- When a user clicks a notification to read it, update `UserNotification.isRead = true` via a background API call.

Verification Plan

Automated / API Verification

1. **Split Delivery Test**: Create a PO for 100 units. Approve a GRN for 40 units. Verify the PO status updates strictly to `Partially Delivered` and the `deliveredQty` is 40.
2. **Three-Way Match Test**: Submit an Invoice for 40 units against the above PO. Verify it results in a `Full Match`. Submit a second Invoice for 10 units. Verify it results in a `Variance` because `AllowedToBill` is now 0.
3. **Notification Isolation Test**: Generate a notification for Manager A and Manager B. Manager A clicks it. Verify `isRead` is `true` for Manager A, but remains `false` for Manager B.

Manual UI Verification

1. Open the Sidebar Notification Dropdown. Verify the new action buttons (`[Approve PO]`) are visible directly inside the popup.
2. Click the action button. Verify it executes immediately, the button turns to "Completed", and the underlying entity (e.g., PO) changes status in the background.
3. Open a `Partially Delivered` PO. Verify the visual ledger (progress bars) accurately shows `Delivered vs Ordered`.